

Introduction to Generalized Linear Models

Iterative Re-weighted Least Square Algorithm

Table of contents

GLM	2
Poisson Regression	5
Logistic Regression	13

GLM

Overview

A **generalized linear model** is just a linear model, but with a modified error distribution that better captures the data generating process that has helped in the creation of your data. For instance, if your responses are successes and failures, which are typically coded as 1s and 0s, it is pretty clear that 1s and 0s are not normally distributed data nor would they have a normal error distribution on their residuals. In cases like these we need to modify our underlying linear modeling distribution to best match what distribution has helped shape our data.

Generalized linear regression model (GLM) and general linear regression are usually treated as different regression methods. The general linear regression is just one special case of GLM. For the general linear regression we can write the model as

$$Y_i = \beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip} + \epsilon_i$$

where $\epsilon_i \stackrel{iid}{\sim} N(0, \sigma^2)$. We treat β and X as constant, then Y_i can be seen has the same distribution as ϵ_i .

For the generalized linear model, (GLM), there is no error term ϵ_i anymore, we directly assume that the response variable Y has **certain distributions** such as Poisson, Bernoulli, Gamma distribution et al. For these distributions we assume the parameter μ i.e $E(Y)$ by some **transformations** are **linear** with predictor variables. These transformations are called **link functions** which can be written as:

$$g(\mu_i) = \beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip}$$

where $g(x)$ is called link function and we usually denote

$$\eta_i = \beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip}, i = 1, \dots, n$$

The exponential family

Assuming that each component of Y has a distribution in the exponential family, taking the form

$$f(y; \theta; \phi) = \exp \left\{ \frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi) \right\} \quad (0.1)$$

for some specific functions $a(\cdot)$, $b(\cdot)$ and $c(\cdot)$. If ϕ is known, this is an exponential-family model with cononical parameter θ . For instance, for the Normal distribution

$$\begin{aligned} f(y; \mu; \sigma^2) &= \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{-\frac{(y - \mu)^2}{2\sigma^2}\right\} \\ &= \exp(\log(2\pi\sigma^2)^{-0.5}) \exp\left\{-\frac{(y^2 - 2y\mu + \mu^2)}{2\sigma^2}\right\} \\ &= \exp\left\{\log(2\pi\sigma^2)^{-0.5} - \frac{(y^2 - 2y\mu + \mu^2)}{2\sigma^2}\right\} \\ &= \exp\left\{\frac{(y\mu - \mu^2/2)}{\sigma^2} - \frac{1}{2}(y^2/\sigma^2 + \log(2\pi\sigma^2))\right\} \end{aligned}$$

So that $\theta = \mu$, $\phi = \sigma^2$ and $a(\phi) = \phi = \sigma^2$, $b(\theta) = \mu^2/2$, $c(y, \phi) = -\frac{1}{2}(y^2/\sigma^2 + \log(2\pi\sigma^2))$

We write $l(\theta; \phi; y) = \log f(y; \theta; \phi)$ for the log-likelihood function which considered as a function of θ , ϕ , and y being given. We have from Equation 0.1 that

$$l(\theta; y) = \frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi)$$

Consider that

$$\frac{\partial l}{\partial \theta} = \frac{y - b'(\theta)}{a(\phi)}$$

and

$$\frac{\partial^2 l}{\partial \theta^2} = -\frac{b''(\theta)}{a(\phi)}$$

We then have

$$E\left(\frac{\partial l}{\partial \theta}\right) = E\left(\frac{y - b'(\theta)}{a(\phi)}\right) = \frac{\mu - b'(\theta)}{a(\phi)}$$

Let $E\left(\frac{\partial l}{\partial \theta}\right) = 0$, then we have

$$E(Y) = b'(\theta)$$

Similarly from, we have

$$\begin{aligned}
 0 &= E\left(\frac{\partial^2 l}{\partial \theta^2}\right) + E\left(\frac{\partial l}{\partial \theta}\right)^2 \\
 &= E\left(-\frac{b''(\theta)}{a(\phi)}\right) + E\left(\frac{y - b'(\theta)}{a(\phi)}\right)^2 \\
 &= -\frac{b''(\theta)}{a(\phi)} + E\left(\frac{y - E(Y)}{a(\phi)}\right)^2 \\
 &= -\frac{b''(\theta)}{a(\phi)} + \frac{\text{Var}(Y)}{a^2(\phi)}
 \end{aligned}$$

So that

$$\text{Var}(Y) = b''(\theta)a(\phi)$$

i Note

The variance of Y is the product of two functions

1. $b''(\theta)$, depends on the canonical parameter θ only and it will be called the variance function
2. $a(\phi)$ is independent of θ and depends only ϕ

i Note

Suppose we have the likelihood function as follows:

$$f(y; \theta; \phi) = \exp \left\{ \frac{y\theta - b(\theta)}{a(\phi)} + c(y, \phi) \right\}$$

Then the mean and variance of Y can be derived from the relations

$$E\left(\frac{\partial l}{\partial \theta}\right) = 0$$

and

$$E\left(\frac{\partial^2 l}{\partial \theta^2}\right) + E\left(\frac{\partial l}{\partial \theta}\right)^2 = 0$$

where $l(\theta; \phi; y)$ is the log-likelihood function.

Examples

Bernoulli:

A Bernoulli random variable Y assigns probability measure p to the point $Y = 1$ and probability measure $1 - p$ to $Y = 0$. More formally, define ν to be counting measure on $\{0, 1\}$, and define the following density function with respect to ν :

$$f(y; p) = p^y(1 - p)^{1-y} = \exp\{y \log(\frac{p}{1-p}) - (-\log(1-p))\}$$

Thus we see that the Bernoulli distribution is an exponential family distribution with:

$$\begin{aligned}\theta &= \log(\frac{p}{1-p}) \\ b(\theta) &= \log\{1 + \exp\{\theta\}\} \\ c(y; \phi) &= 1 \\ \phi &= 1\end{aligned}$$

Poisson:

Let Y be a Poisson random variable with parameter $\mu > 0$. The functional form of the Poisson is

$$f(y; \mu) = \frac{e^{-\mu} \mu^y}{y!}$$

It can be written in exponential family form as

$$\begin{aligned}f(y; \mu) &= \exp\{-\mu\} \exp\{\log \mu^y\} \exp\{\log((y!)^{-1})\} \\ &= \exp\{\frac{y \log \mu - \exp(\log \mu)}{1} - \log(y!)\}\end{aligned}$$

Thus, we have

$$\begin{aligned}\theta &= \log \mu \\ b(\theta) &= \exp(\theta) \\ c(y; \phi) &= -\log(y!) \\ \phi &= 1 \\ a(\phi) &= 1\end{aligned}$$

Poisson Regression

Poisson Regression in R

Remember that the Poisson distribution pertains to values that are non-negative integers only. The quintessential example of Poisson data involves count observations, as counts cannot be negative and are usually whole numbers. This distribution is characterized by a single parameter, denoted as μ , which serves as both the mean and the variance. A Poisson regression

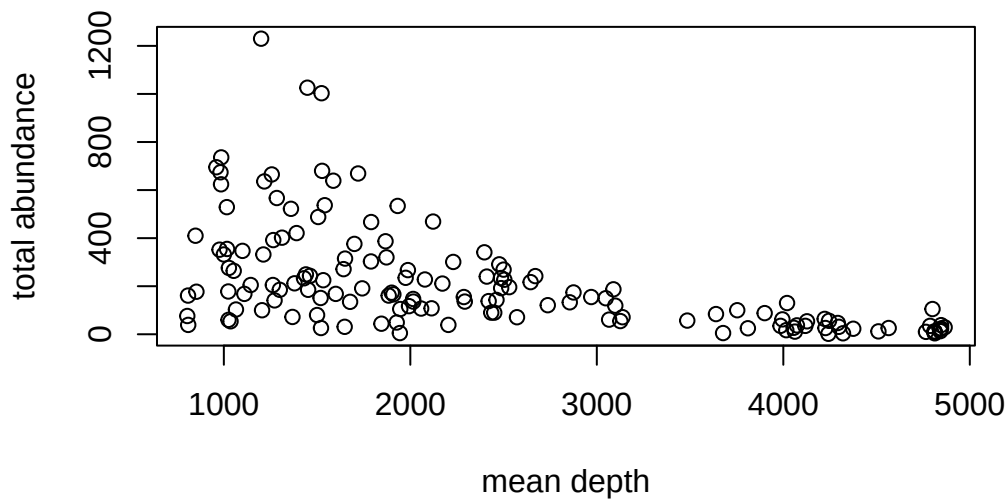
uses Log link (and therefore the coefficients need to be exponentiated to return them to the natural scale)

Example: Using the fishing data in the COUNT library, let's model the relationship between total abundance (totabund) and mean depth (meandepth). Total abundance are counts, and we might hypothesize that abundances of fishes decreases with increasing depth.

```
library(COUNT)
data(fishing)
head(fishing)
```

	site	totabund	density	meandepth	year	period	sweptarea
1	1	76	0.002070281	804	1978	1977-1989	36710.00
2	2	161	0.003519799	808	2001	2000-2002	45741.25
3	3	39	0.000980515	809	2001	2000-2002	39775.00
4	4	410	0.008039216	848	1979	1977-1989	51000.00
5	5	177	0.005933375	853	2002	2000-2002	29831.25
6	6	695	0.021800501	960	1980	1977-1989	31880.00

```
plot(x = fishing$meandepth, y = fishing$totabund,
     ylab="total abundance", xlab="mean depth")
```



```
pois.glm <- glm(totabund ~ meandepth, data = fishing, family = poisson)
summary(pois.glm)
```

Call:

```
glm(formula = totabund ~ meandepth, family = poisson, data = fishing)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	6.647e+00	1.269e-02	523.96	<2e-16 ***
meandepth	-6.309e-04	6.659e-06	-94.74	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

Null deviance: 28050 on 146 degrees of freedom
 Residual deviance: 15778 on 145 degrees of freedom
 AIC: 16754

Number of Fisher Scoring iterations: 5

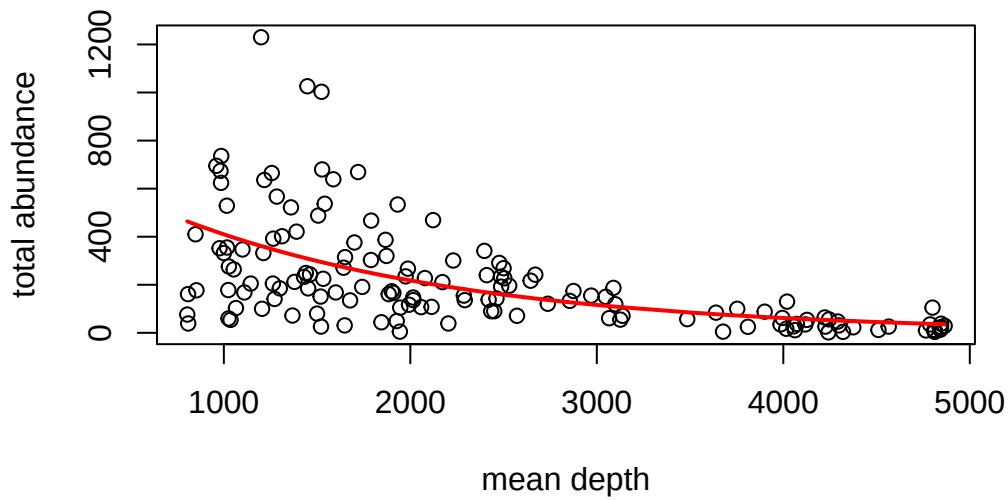
The estimated coefficients don't like numbers we used in the model, but we have to remember that these are estimates on the log scale, so if we back-transform them, we should recover more meaningful numbers.

```
exp(coef(pois.glm))
```

(Intercept)	meandepth
770.1424771	0.9993693

The slope may be a little harder to interpret, but the intercept of 770 makes a lot of sense given the plot. Finally, we may want to plot the model.

```
beta0 <- coef(pois.glm)[1]
beta1 <- coef(pois.glm)[2]
muhat <- exp(fishing$meandepth*beta1+beta0)
plot(x = fishing$meandepth, y = fishing$totabund,
     ylab="total abundance", xlab="mean depth")
lines(x = fishing$meandepth, y=muhat, lwd=2, col="red")
```



Following is the interpretation for the parameter estimates:

- $\exp(\beta_0)$ = effect on the mean μ , when $X = 0$
- $\exp(\beta_1)$ = with every unit increase in X , the predictor variable has multiplicative effect of $\exp(\beta_1)$ on the mean of Y
 - $\beta_1 = 0$, then $\exp(\beta_1) = 1$, and the expected count is $\exp(\beta_0)$ and, Y and X are not related.
 - If $\beta_1 > 0$, then $\exp(\beta) > 1$, and the expected count is $\exp(\beta_1)$ times larger than when $X = 0$
 - If $\beta_1 < 0$, then $\exp(\beta) < 1$, and the expected count is $\exp(\beta_1)$ times smaller than when $X = 0$

Predicting From The Model

```
newdata <- data.frame(meandepth = 1500)
predict(pois.glm, newdata = newdata, type = "response")
```

1
298.9282

Estimate Coefficients for Poisson regression

To calculate the β and its confidence interval, maximum likelihood estimating (MLE) method is often used. Often, we cannot find an explicit solution for the MLE, therefore this method is usually combined with a Newton-Raphson algorithm which leads to so called **iterative re-weighted least square, IWLS/IRLS** method. Let suppose our responsive variable Y has a Poisson distribution, then we can write:

$$f(y_i; \mu_i) = \frac{e^{-\mu_i} \mu_i^{y_i}}{y_i!}, i = 1, \dots, n$$

The likelihood function is:

$$L(\mu_i; y_i) = \prod_{i=1}^n f(y_i; \mu_i) = \prod_{i=1}^n \frac{e^{-\mu_i} \mu_i^{y_i}}{y_i!}$$

Then the log likelihood function is given by:

$$l = \log L = - \sum_{i=1}^n \mu_i + \sum_{i=1}^n y_i \log \mu_i - \sum_{i=1}^n \log y_i! \quad (0.2)$$

We choose log as the **link function** and we have:

$$g(\mu_i) = \log(\mu_i) = \beta_1 + \beta_2 x_{i2} + \dots + \beta_p x_{ip} \quad (0.3)$$

Without loss of generality, we denote the design matrix as follows:

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1p} \\ x_{21} & x_{22} & \dots & x_{2p} \\ \dots & \dots & \dots & \dots \\ x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}$$

Let $\mathbf{x}_i$ is the row vector $[x_{i1}, x_{i2}, \dots, x_{ip}]^T$ and $\beta = [\beta_1, \dots, \beta_p]^T$ where i is the i th subject in our data set and p is the number of predictors.

From Equation 0.3, we get

$$\mu_i = e^{\mathbf{x}_i^T \beta} \quad (0.4)$$

and put Equation 0.4 back to Equation 0.2

$$l = \log L = - \sum_{i=1}^n e^{\mathbf{x}_i^T \beta} + \sum_{i=1}^n y_i \log(e^{\mathbf{x}_i^T \beta}) - \sum_{i=1}^n \log y_i!$$

Taking derivatives in term of β_j where $j = 1, 2, \dots, p$, we get:

$$\begin{aligned}
\frac{\partial l}{\partial \beta_j} &= - \sum_{i=1}^n x_{ij}(e^{\mathbf{x}_i^T \beta}) + \sum_{i=1}^n y_i x_{ij} \\
&= \sum_{i=1}^n x_{ij}(y_i - e^{\mathbf{x}_i^T \beta}) \\
&= \sum_{i=1}^n x_{ij}(y_i - \mu_i), j = 1, \dots, p
\end{aligned}$$

We can have $1, 2, \dots, p$ such equation and set them to zero, as long as we have number of subjects $n > p$ we can solve these equations for $\beta_1, \dots, \beta_p$. We can write above equations as matrix format

$$\nabla l(\beta; y, x) = X^T(Y - e^{\mathbf{X}\beta}) \quad (0.5)$$

And we can use the Newton-Raphson method to solve the equation:

$$\nabla l(\beta; y, x) = X^T(Y - e^{\mathbf{X}\beta}) = \mathbf{0} \quad (0.6)$$

To use the Newton-Raphson method, we need to take derivative of the score function Equation 0.5 again. Then, we have

$$\begin{aligned}
\frac{\partial}{\partial \beta_j} \left\{ \frac{\partial l}{\partial \beta_j} \right\} &= \frac{\partial}{\partial \beta_j} \left\{ - \sum_{i=1}^n x_{ij}(e^{\mathbf{x}_i^T \beta}) + \sum_{i=1}^n y_i x_{ij} \right\} \\
&= - \sum_{i=1}^n x_{ij}^2 (e^{\mathbf{x}_i^T \beta})
\end{aligned}$$

and

$$\begin{aligned}
\frac{\partial}{\partial \beta_k} \left\{ \frac{\partial l}{\partial \beta_j} \right\} &= \frac{\partial}{\partial \beta_k} \left\{ - \sum_{i=1}^n x_{ij}(e^{\mathbf{x}_i^T \beta}) + \sum_{i=1}^n y_i x_{ij} \right\} \\
&= - \sum_{i=1}^n x_{ij} x_{ik} (e^{\mathbf{x}_i^T \beta})
\end{aligned}$$

We can write above equations as matrix format

$$\nabla^2 l(\beta; y, x) = -X^T W X$$

where W is a $n \times n$ diagonal matrix with $\exp x_i^T \beta$ for the i diagonal element. Then, we can use Newton-Raphson method to solve equation Equation 0.6.

$$\begin{aligned}
\beta^{(m+1)} &= \beta^{(m)} - (\nabla^2 l(\beta^{(m)}; y, x))^{-1} \nabla l(\beta^{(m)}; y, x) \\
&= \beta^{(m)} + (X^T W^{(m)} X)^{-1} (X^T (Y - e^{\mathbf{X}\beta^{(m)}}))
\end{aligned} \quad (0.7)$$

Let $z^{(m)} = W^{(m)-1}(Y - e^{X\beta^{(m)}})$ and we have $W^{(m)}z^{(m)} = (Y - e^{X\beta^{(m)}})$. Then Equation 0.7 becomes

$$\beta^{(m+1)} = \beta^{(m)} + (X^T W^{(m)} X)^{-1} (X^T W^{(m)} z^{(m)})$$

i Note

$(X^T W^{(m)} X)^{-1} (X^T W^{(m)} z^{(m)})$ is exactly β for a weighted least squares regression of $z^{(m)}$ on X , i.e we can use a weighted least squares regression method to solve this part.

The hand-made IRLS algorithm

We showed how to develop the Iterative re-weighted least squares(IRLS) algorithm from the Newton-Raphson algorithm.

```
n <- length(fishing$meandepth)
# design matrix
X <- cbind(rep(1,n), fishing$meandepth)
y <- as.vector(fishing$totabund)
MyIRLS.poisson<-function(y, X, init.beta, tol = 1e-8){
  change <- Inf
  beta.m <- init.beta
  n.iter <- 1
  n <- nrow(X)
  while(change > tol) {
    eta <- X %*% beta.m # linear predictor, X*Beta
    w<-diag(as.vector(exp(eta)), nrow = n, ncol = n)
    beta.new <- beta.m + solve(t(X)%*%w%*%X)%*%t(X)%*%(y-exp(eta))
    change <- sqrt(sum((beta.new - beta.m)^2))
    beta.m <- beta.new
    n.iter <- n.iter + 1
  }
  return(list(beta = beta.new, n.iter = n.iter))
}
fit1 <- MyIRLS.poisson(y = y, X = X, init.beta = c(0.01,0.02) )
fit1$beta
```

```
      [,1]
[1,]  6.6465755329
[2,] -0.0006309148
```

```
coef(pois.glm)
```

```
(Intercept)      meandepth  
6.6465755329 -0.0006309148
```

```
fit1$n.iter
```

```
[1] 98
```

Next we go a little bit further to do a so called IRLS method, i.e we use a weighted least square regression model to update the coefficients.

```
MyIRLS.poisson1<-function(y, X, init.beta, tol = 1e-8){  
  change <- Inf  
  beta.m <- init.beta  
  n.iter <- 1  
  n <- nrow(X)  
  while(change > tol) {  
    eta <- X %*% beta.m # linear predictor, X*Beta  
    w <- diag(as.vector(exp(eta)), nrow = n, ncol = n)  
    weight <- exp(eta)  
    z<-solve(w)%*%(y-exp(eta))  
    beta.new <- beta.m + lm(z~X-1, weights = weight )$coef  
    #beta.new<-beta.m+solve(t(X)%*%w%*%X)%*%t(X)%*%(y-exp(eta))  
    change <- sqrt(sum((beta.new - beta.m)^2))  
    beta.m <- beta.new  
    n.iter <- n.iter + 1  
  }  
  return(list(beta = beta.new, n.iter = n.iter))  
}  
fit2 <- MyIRLS.poisson1(y = y, X = X, init.beta = c(3, 0) )  
fit2$beta
```

```
      X1      X2  
6.6465755329 -0.0006309148
```

```
coef(pois.glm)
```

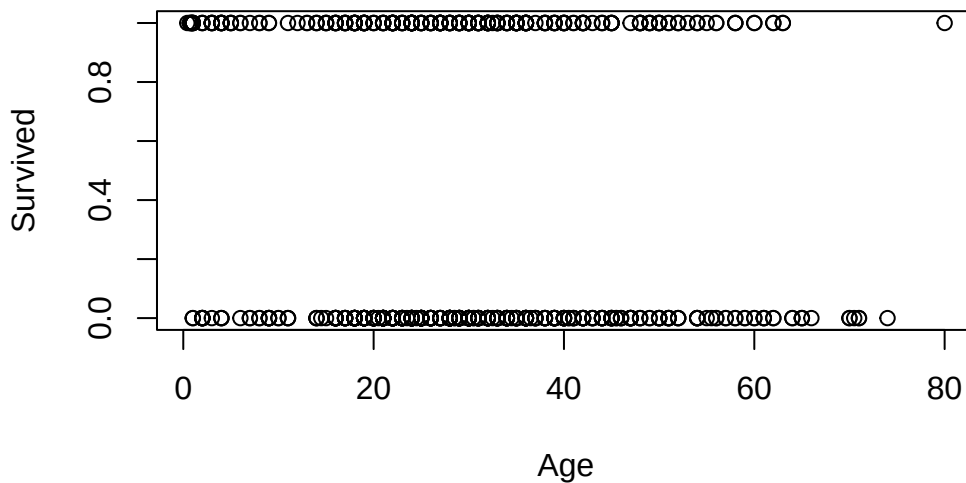
```
(Intercept)      meandepth  
6.6465755329 -0.0006309148
```

Logistic Regression

Binomial regression is for binomial data that have some **number** of successes or failures from some number of trials. Let's focus on the most common application of the binomial regression which is that when the number of trials is 1, which is often called logistic regression. The application of this model is when we have 1s and 0s as our outcomes, which often represent successes or failures, presence or absence, or any other binary outcome.

An Example

```
MyTitanic0 <- titanic::titanic_train
MyTitanic <- na.omit(MyTitanic0)
Y <- MyTitanic$Survived
n <- length(Y)
x1 <- MyTitanic$Age
plot(x = x1, y = Y, xlab="Age", ylab="Survived")
```



Estimate Coefficients for Logistic regression

Let suppose the response variable Y has a Bernoulli distribution then we write

$$f(y_i, p_i) = p_i^{y_i} (1 - p_i)^{1-y_i}, i = 1, \dots, n$$

The likelihood function is:

$$\begin{aligned} L &= \prod_{i=1}^n f(y_i, p_i) \\ &= \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i} \end{aligned}$$

The log likelihood function is:

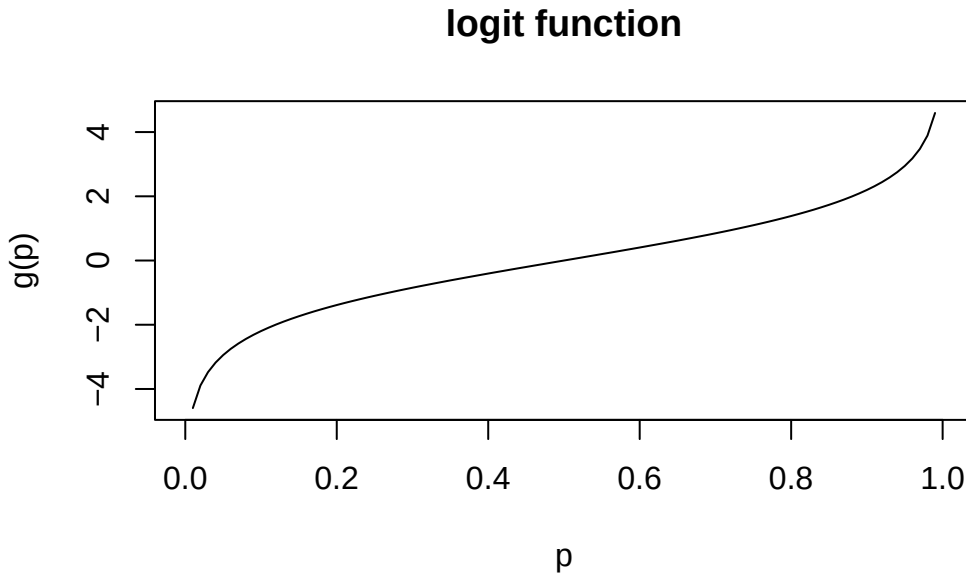
$$l = \log L = \sum_{i=1}^n y_i \log p_i + \sum_{i=1}^n (1 - y_i) \log(1 - p_i) \quad (0.8)$$

The link function for logistic regression model is called logit function i.e :

$$g(p_i) = \text{logit}(p_i) = \log\left(\frac{p_i}{1-p_i}\right) = \beta_1 X_{i1} + \beta_2 X_{i2} + \dots + \beta_p X_{ip} = x_i^T \beta \quad (0.9)$$

The link function, logit function, can be expressed as follows:

```
curve( expr = log(x/(1-x)), from = 0, to = 1,
       ylab="g(p)", xlab="p", main="logit function")
```



From Equation 0.9, we have

$$\frac{p_i}{1-p_i} = e^{x_i^T \beta}$$

and

$$\begin{aligned} p_i &= \frac{e^{x_i^T \beta}}{1 + e^{x_i^T \beta}} \\ &= \frac{1}{1 + e^{-x_i^T \beta}} \end{aligned}$$

Put Equation Equation 0.10 to Equation 0.8, then we have

$$\begin{aligned} l(\beta; Y, X) &= \sum_{i=1}^n y_i \log p_i + \sum_{i=1}^n (1 - y_i) \log(1 - p_i) \\ &= \sum_{i=1}^n y_i \log\left(\frac{1}{1 + e^{-x_i^T \beta}}\right) + \sum_{i=1}^n (1 - y_i) \log\left(1 - \frac{1}{1 + e^{-x_i^T \beta}}\right) \\ &= \sum_{i=1}^n \left\{ y_i (x_i^T \beta) - x_i^T \beta - \log(1 + e^{-x_i^T \beta}) \right\} \end{aligned}$$

Taking derivatives in term of β_j where $j = 1, 2, \dots, p$, we get:

$$\begin{aligned} \frac{\partial l}{\partial \beta_j} &= \sum_{i=1}^n \left\{ y_i x_{ij} - x_{ij} + \frac{x_{ij}(e^{-x_i^T \beta})}{1 + e^{-x_i^T \beta}} \right\} \\ &= \sum_{i=1}^n x_{ij} \left\{ y_i - 1 + \frac{e^{-x_i^T \beta}}{1 + e^{-x_i^T \beta}} \right\} \\ &= \sum_{i=1}^n x_{ij} \left\{ y_i + \frac{e^{-x_i^T \beta} - (1 + e^{-x_i^T \beta})}{1 + e^{-x_i^T \beta}} \right\} \\ &= \sum_{i=1}^n x_{ij} \left\{ y_i - \frac{1}{1 + e^{-x_i^T \beta}} \right\}, j = 1, \dots, p \end{aligned}$$

We write these equations as matrix format:

$$\begin{aligned} \nabla l(\beta; y, x) &= X^T (\mathbf{Y} - \mathbf{p}) \\ &= X^T \left(\mathbf{Y} - \frac{1}{1 + e^{-X^T \beta}} \right) \end{aligned}$$

where $p_i = 1/(1 + e^{-x_i^T \beta})$, $i = 1, \dots, n$. In order to use Newton-Rapson method, we need to

take derivative in term of β for $\nabla l(\beta; y, x)$ to get

$$\begin{aligned}\nabla^2 l(\beta; y, x) &= -X^T \left\{ -\frac{e^{-X^T \beta}}{(1 + e^{-X^T \beta})^2} \right\} X \\ &= -X^T \left\{ \frac{1}{1 + e^{-X^T \beta}} \frac{e^{-X^T \beta}}{1 + e^{-X^T \beta}} \right\} X \\ &= -X^T [p(1 - p)] X \\ &= -X^T W X\end{aligned}$$

where W is a $n \times n$ diagonal matrix with the Bernoulli variance $p_i(1 - p_i)$ as the i th diagonal element. We then can use Newton-Raphson method to solve:

$$\nabla l(\beta; y, x) = 0$$

Using Newton-Raphson method we have:

$$\begin{aligned}\beta^{(m+1)} &= \beta^{(m)} - (\nabla^2 l(\beta^{(m)}; y, x))^{-1} \nabla l(\beta^{(m)}; y, x) \\ &= \beta^{(m)} + (X^T W^{(m)} X)^{-1} (X^T (Y - p^{(m)})) \\ &= \beta^{(m)} + (X^T W^{(m)} X)^{-1} (X^T W^{(m)} Z^{(m)}) \\ &= [(X^T W^{(m)} X)^{-1} (X^T W^{(m)} X)] \beta^{(m)} + (X^T W^{(m)} X)^{-1} (X^T W^{(m)} Z^{(m)}) \\ &= (X^T W^{(m)} X)^{-1} [(X^T W^{(m)} X) \beta^{(m)} + X^T W^{(m)} Z^{(m)}] \\ &= (X^T W^{(m)} X)^{-1} X^T W^{(m)} [X \beta^{(m)} + Z^{(m)}]\end{aligned}$$

where $Z^{(m)} = W^{-1}(Y - 1/(1 + e^{-X^T \beta}))$. From the last equation, we can see that it is the coefficients for the weighted least squares regression $X \beta^{(m)} + Z^{(m)}$ on X .

```
MyIRLS_logistic <- function(Y, X, init.beta, tol = 1e-8){
  change <- Inf
  beta.m <- init.beta
  n <- nrow(X)
  q <- dim(X)[2]
  n.iter <- 1
  while(change > tol) {
    eta <- X %%% matrix(beta.m, nrow = q)      # linear predictor
    p <- 1/(1+exp(-eta))
    w<-diag(as.vector(p*(1-p)),nrow=n, ncol= n )#diagonal matrix
    weight <- p*(1-p) #vector weight for wls
    z <- solve(w)%%(Y-p)
    beta.new <- beta.m + solve(t(X)%%w%%X)%%t(X)%%w%%z
    #z.new <- X%%beta.m + z
  }
}
```



```

        #beta.new <- lm(z.new ~ X - 1, weights = weight)$coef # WLS regression
        change <- sqrt(sum((beta.new - beta.m)^2))
        beta.m <- beta.new
        n.iter <- n.iter + 1
    }
    return(list(beta=beta.new, n.iter=n.iter))
}
MyTitanic0 <- titanic::titanic_train
MyTitanic <- na.omit(MyTitanic0)
Y <- MyTitanic$Survived
n <- length(Y)
x1 <- MyTitanic$Age
X <- cbind(rep(1, n), x1)
df <- data.frame(Y, age=x1)
fit3 <- glm(Y ~ age , data = df, family = "binomial"(link = "logit"))
coefficients(fit3)

```

```

(Intercept)      age
-0.05672364 -0.01096345

```

```
MyIRLS_logistic(Y = Y, X = X, init.beta = c(-0.1, 0))
```

```

$beta
      [,1]
      -0.05672364
x1 -0.01096345

```

```

$n.iter
[1] 5

```

Using our self-implemented IRLS function, we observe identical results to those obtained with the glm function.